

Replicação do *Reflective Gaussian Splatting* e Propostas para a Modelagem de Visibilidade em Superfícies Reflexivas

[Nome completo] Pimenta

Centro de Informática

Universidade Federal de Pernambuco

Recife, Brasil

jpmp150404@gmail.com

[Nome do orientador]

Centro de Informática

Universidade Federal de Pernambuco

Recife, Brasil

[e-mail]

Resumo—A reconstrução de superfícies reflexivas continua sendo um ponto fraco dos métodos de síntese de novas vistas baseados em Gaussian Splatting, pois a aparência dependente da direção de observação é aprendida sem um modelo físico explícito da luz refletida. O *Reflective Gaussian Splatting* (Ref-Gaussian) aborda o problema com renderização diferida fisicamente baseada sobre Gaussianas 2D e com um termo de inter-reflexão calculado por ray tracing sobre uma malha extraída periodicamente da cena. Este relatório descreve o método, identifica suas limitações quanto à aproximação de visibilidade e ao custo da extração de malha, e propõe duas linhas de investigação para a dissertação. Como primeiro passo experimental, o código oficial foi portado e executado em uma GPU de notebook sob Windows, e o baseline foi replicado no conjunto Shiny Blender Synthetic. Na cena toaster, obteve-se PSNR de 27,51 dB, SSIM de 0,948 e LPIPS de 0,074, contra 28,05 dB, 0,948 e 0,074 reportados no artigo original.

Index Terms—Gaussian Splatting, superfícies reflexivas, renderização fisicamente baseada, inter-reflexão, síntese de novas vistas.

I. INTRODUÇÃO

O 3D Gaussian Splatting (3DGS) [1] representa uma cena como um conjunto de Gaussianas anisotrópicas rasterizadas de forma diferenciável, alcançando qualidade comparável à dos campos de radiância neurais com renderização em tempo real. Sua aparência dependente de vista, porém, é modelada por harmônicos esféricos de baixa ordem, o que é insuficiente para superfícies especulares: reflexos nítidos são aprendidos como geometria falsa “atrás” do objeto e as normais resultantes ficam incorretas.

Trabalhos recentes incorporam modelos de sombreamento ao 3DGS. O GaussianShader [4] associa a cada Gaussiana uma função de sombreamento simplificada; o 3DGS-DR [5] adia o cálculo da reflexão para o espaço da imagem; e o Ref-Gaussian [3], adotado aqui como *baseline*, combina Gaussianas 2D [2], renderização diferida fisicamente baseada e um termo de inter-reflexão. O Ref-Gaussian supera os métodos anteriores no Shiny Blender [6] em todas as cenas (Tabela I).

Este relatório tem dois objetivos: (i) descrever o método base e as linhas de investigação propostas para a dissertação (Seção II); e (ii) replicar os resultados do baseline em hardware

Tabela I

PSNR NO SHINY BLENDER REPORTADO POR YAO ET AL. [3] PARA MÉTODOS ANTERIORES, COMO CONTEXTO PARA O BASELINE.

Método	Ball	Car	Coffee	Helmet	Teapot	Toaster
3DGS [1]	27,65	27,26	32,30	28,22	45,71	20,99
2DGS [2]	25,97	26,38	32,31	27,42	44,97	20,42
Ref-NeRF [6]	33,16	30,44	33,99	29,94	45,12	26,12
GShader [4]	30,99	27,96	32,39	28,32	45,86	26,28
3DGS-DR [5]	33,43	30,48	34,53	31,44	47,04	26,76
Ref-Gaussian [3]	37,01	31,04	34,63	32,32	47,16	28,05

de consumo, condição necessária antes de propor modificações (Seções III e IV).

II. SOLUÇÃO PROPOSTA

A. Método base: *Reflective Gaussian Splatting*

A Fig. 1 resume o pipeline do Ref-Gaussian. Seus componentes são descritos a seguir.

Representação. A cena é composta por *surfels* Gaussianos 2D [2], discos orientados definidos por centro, dois vetores tangentes, escalas e opacidade. A normal de cada primitiva é o vetor perpendicular ao seu plano, o que fornece geometria de superfície mais precisa que a das Gaussianas 3D. Cada primitiva carrega ainda os atributos de material do modelo Disney/PBR: albedo $\mathbf{a}$, metalicidade m e rugosidade r .

Renderização diferida. Em vez de sombrear cada Gaussiana e depois combinar as cores, os atributos ($\mathbf{a}$, m , r , normal $\mathbf{n}$ e profundidade) são primeiro combinados por *alpha blending* em mapas por pixel. O sombreamento é então avaliado uma única vez por pixel, o que gera reflexos mais nítidos e consistentes. A radiância de saída na direção ω_o é dividida em um termo difuso e um especular:

$$L(\omega_o) = L_d + L_s, \quad L_d = (1-m) \mathbf{a} \frac{1}{\pi} \int_{\Omega} L_i(\omega_i) (\omega_i \cdot \mathbf{n}) d\omega_i. \quad (1)$$

O termo especular usa a aproximação *split-sum* [8]:

$$L_s \approx (\mathbf{F}_0 A(\mathbf{n} \cdot \omega_o, r) + B(\mathbf{n} \cdot \omega_o, r)) \tilde{L}(\omega_r, r), \quad (2)$$

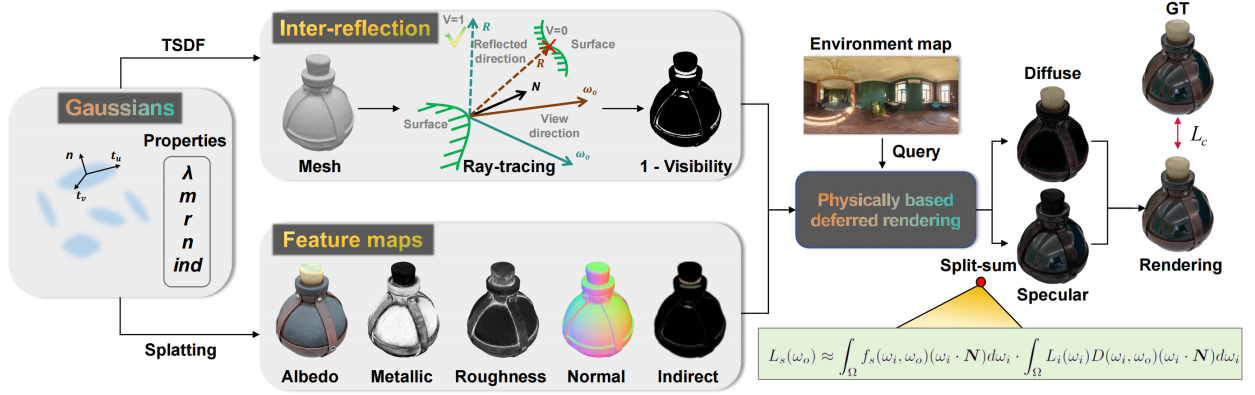


Figura 1. Pipeline do Ref-Gaussian: Gaussianas 2D com atributos de material são rasterizadas em mapas por pixel (renderização diferida); o sombreamento combina iluminação direta de um mapa de ambiente com inter-reflexão fundamentada nas próprias Gaussianas. Fonte: Yao et al. [3].

Tabela II

REPLICAÇÃO DO REF-GAUSSIAN NO SHINY BLENDER SYNTHETIC: VALORES REPORTADOS NO ARTIGO [3] E OBTIDOS NESTE TRABALHO (RTX 4050 LAPTOP); Δ É A DIFERENÇA DE PSNR DA REPLICAÇÃO EM RELAÇÃO AO ARTIGO. “-” INDICA TREINO EM ANDAMENTO.

Métrica	Origem	Ball	Car	Coffee	Helmet	Teapot	Toaster	Média
PSNR $\uparrow$	Artigo	37,01	31,04	34,63	32,32	47,16	28,05	35,04
	Replicação	-	-	-	-	-	27,51	-
	Δ	-	-	-	-	-	-0,54	-
SSIM $\uparrow$	Artigo	0,981	0,964	0,976	0,971	0,998	0,948	0,973
	Replicação	-	-	-	-	-	0,948	-
LPIPS $\downarrow$	Artigo	0,098	0,033	0,076	0,049	0,006	0,074	0,056
	Replicação	-	-	-	-	-	0,074	-

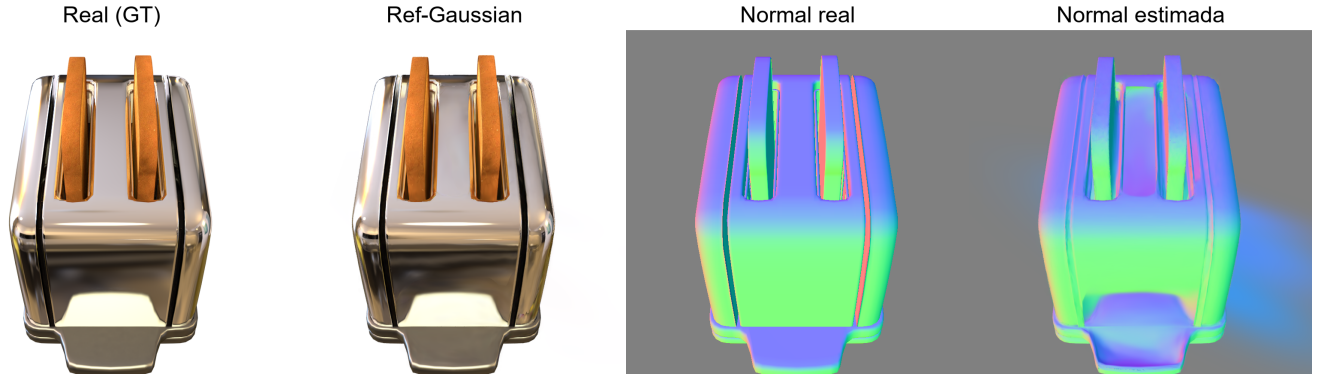


Figura 2. Resultado qualitativo na cena *toaster* (vista de teste 0) após 50.000 iterações. Da esquerda para a direita: imagem real, renderização do Ref-Gaussian, normal real e normal estimada.

com $\mathbf{F}_0 = 0,04(1-m) + m\mathbf{a}$, termos A e B pré-integrados da BRDF em uma tabela, e $\tilde{L}$ a iluminação pré-filtrada amostrada na direção de reflexão ω_r em um mapa de ambiente cúbico treinável com níveis de mipmap indexados pela rugosidade.

Inter-reflexão fundamentada em Gaussianas. O mapa de ambiente representa apenas luz vinda do infinito; reflexos de uma parte do objeto sobre outra exigem um termo indireto. O Ref-Gaussian separa a componente especular em

$$L_s = V L_s^{\text{dir}} + (1 - V) L_s^{\text{ind}}, \quad V \in \{0, 1\}, \quad (3)$$

em que a visibilidade V é obtida lançando um único raio

na direção de reflexão ω_r contra uma malha extraída das Gaussianas por fusão TSDF. A radiância indireta L_s^{ind} é obtida da própria representação Gaussiana da cena. A malha é reextraída periodicamente durante o treino.

Propagação de normais sensível ao material. Para corrigir normais em regiões altamente especulares, as Gaussianas com $m \geq 0,02$ e $r \leq 0,1$ têm suas escalas periodicamente ampliadas, propagando normais corretas para as vizinhas.

Treinamento em estágios. O treino começa com sombreamento por Gaussianas, que estabiliza a geometria, e depois

passa à renderização diferida. A função de perda é

$$\mathcal{L} = \mathcal{L}_c + \lambda_n \mathcal{L}_n + \lambda_s \mathcal{L}_{\text{smooth}}, \quad (4)$$

com $\mathcal{L}_c$ a perda fotométrica, $\mathcal{L}_n$ a consistência entre normais renderizadas e derivadas da profundidade [2] e $\mathcal{L}_{\text{smooth}}$ uma suavização de normais. O artigo usa $\lambda_n = 0,05$ e $\lambda_s = 1,0$.

B. Limitações identificadas

- 1) **Visibilidade pontual.** A Eq. (3) testa a oclusão apenas na direção de reflexão especular perfeita e usa esse único teste para todo o lóbulo especular. Os próprios autores apontam que isso introduz ruído na estimativa de materiais. O erro tende a crescer com a rugosidade, quando o lóbulo se alarga.
- 2) **Dependência de malha extraída.** O ray tracing exige reconstruir uma malha por TSDF a cada poucos milhares de iterações e reconstruir a estrutura de aceleração (BVH). Isso adiciona custo, cria uma segunda representação geométrica que pode divergir das Gaussianas e torna a visibilidade não diferenciável em relação a elas.
- 3) **Escopo.** O método é desenhado para cenas estáticas e predominantemente especulares; os autores reconhecem desempenho inferior em cenas não especulares.

C. Linhas de investigação propostas

As propostas a seguir são hipóteses de trabalho para a dissertação e serão avaliadas contra o baseline replicado neste relatório.

P1 – Visibilidade sobre o lóbulo especular. Substituir o teste binário único por uma estimativa da visibilidade média sobre o lóbulo da BRDF:

$$\bar{V} \approx \frac{1}{K} \sum_{k=1}^K V(\omega_k), \quad \omega_k \sim D_{\text{GGX}}(\cdot; \omega_r, r), \quad (5)$$

com poucas direções ω_k obtidas por amostragem por importância da distribuição GGX, de forma estratificada e com K adaptativo à rugosidade ($K = 1$ recupera o baseline em superfícies espelhadas). Para manter o custo baixo, serão avaliadas reutilização temporal das amostras entre iterações e aproximações por cone. A avaliação usará PSNR/SSIM/LPIPS, o erro de albedo e rugosidade nas cenas sintéticas com material conhecido, e uma ablação de K contra tempo de treino.

P2 – Visibilidade direta a partir das primitivas. Eliminar a extração de malha calculando a oclusão diretamente sobre as Gaussianas, acumulando a transmitância ao longo do raio refletido com uma estrutura de aceleração sobre envelopes das primitivas, na linha do ray tracing de Gaussianas [9]. Além de remover o custo do TSDF, essa formulação permite uma visibilidade contínua e potencialmente diferenciável em relação à geometria. As métricas são as mesmas de P1, mais o tempo total de treino e o consumo de memória.

P3 – Extensão dinâmica (trabalho futuro). Estender a representação por Gaussianas 2D a objetos reflexivos em movimento, tratando a coerência temporal dos reflexos. Esta linha depende dos resultados de P1/P2 e fica como perspectiva.

III. EXPERIMENTOS

O objetivo desta etapa é replicar o baseline com o código oficial antes de qualquer modificação.

A. Ambiente e reprodutibilidade

Os experimentos usam o código oficial do Ref-Gaussian (commit af7691a, mar. 2025) em um notebook com GPU NVIDIA GeForce RTX 4050 Laptop (6 GB), Windows 11, CUDA 11.8, PyTorch 2.0.0 e Python 3.8. O código foi desenvolvido para Linux, e as extensões CUDA (rasterizador de *surfels*, *simple-knn*, codificador de cubemap e o ray tracer baseado em BVH) precisaram de ajustes para compilar com o MSVC 14.44 (Visual Studio 2022), mais recente que o oficialmente suportado pelo CUDA 11.8. Os principais ajustes foram: permitir o compilador não suportado; incluir explicitamente cabeçalhos que o GCC inclui de forma transitiva; ativar o modo de conformidade do MSVC (/permissive-, /Zc:__cplusplus), necessário para compilar a biblioteca Eigen junto com o nvcc; e impedir a inclusão do pybind11 nos arquivos CUDA. Nenhum desses ajustes altera o código-fonte do método; todos foram feitos por meio de flags de compilação, registradas em scripts no repositório do projeto.

B. Conjunto de dados

Nesta etapa usa-se o Shiny Blender Synthetic [6], com seis objetos sintéticos de materiais altamente especulares (*ball*, *car*, *coffee*, *helmet*, *teapot* e *toaster*). Cada cena tem 100 vistas de treino e 200 de teste, em resolução 800×800 e fundo branco. Os conjuntos Glossy Synthetic [7] e Shiny Blender Real serão usados nas próximas etapas.

C. Protocolo

O treino segue os argumentos do script `train.sh` do repositório oficial, com a configuração padrão do código: 50.000 iterações, sombreado por Gaussiana até a iteração 18.000, inter-reflexão a partir da iteração 20.000, extração de malha (resolução 512) a cada 2.000 iterações e densificação até a iteração 25.000. As cenas *helmet* e *ball* usam $\lambda_s = 1,0$, como no script oficial. Esse cronograma difere do descrito no artigo (18.000 iterações iniciais mais 40.000 com renderização diferida, extração a cada 3.000 passos); optou-se pela configuração publicada no código.

As métricas são as do artigo, calculadas pelo script de avaliação oficial sobre as 200 vistas de teste: PSNR, SSIM e LPIPS (VGG). Reportam-se também a taxa de renderização (FPS) e o tempo de treino.

IV. RESULTADOS E DISCUSSÃO

Métricas. A Tabela II compara os resultados obtidos com os do artigo. Na cena *toaster*, a mais difícil do conjunto para todos os métodos (Tabela I), o SSIM e o LPIPS replicados coincidem com os reportados até a terceira casa decimal, e o PSNR fica 0,54 dB abaixo. A diferença é pequena diante da variação esperada entre execuções de métodos com treino estocástico, e há fatores conhecidos que podem explicá-la: GPU, versão do compilador e não determinismo das operações

Tabela III
CUSTO NA RTX 4050 LAPTOP: TEMPO DE TREINO (50.000 ITERAÇÕES) E
TAXA DE RENDERIZAÇÃO NA AVALIAÇÃO.

Cena	Tempo de treino	FPS
<i>ball</i>	—	—
<i>car</i>	—	—
<i>coffee</i>	—	—
<i>helmet</i>	—	—
<i>teapot</i>	—	—
<i>toaster</i>	1h46min	38,9

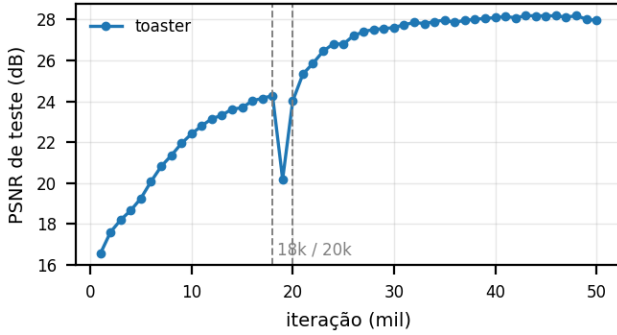


Figura 3. PSNR nas vistas de teste ao longo do treino, para as cenas concluídas. As linhas tracejadas marcam o fim do sombreamento por Gaussiana (18 mil iterações) e o início da inter-reflexão (20 mil).

atômicas na rasterização. Há também a diferença de cronograma de treino descrita na Seção III. Mesmo 0,54 dB abaixo, o resultado supera em 0,75 dB o melhor método anterior nessa cena (3DGS-DR). As demais cenas estão em treino e completarão a tabela na próxima entrega.

Dinâmica do treino. A Fig. 3 mostra a evolução do PSNR de teste, medido pelo próprio script de treino a cada mil iterações. Na cena *toaster*, o estágio de sombreamento por Gaussiana leva o PSNR a cerca de 24,3 dB na iteração 18 mil. A troca para a renderização diferida derruba o valor temporariamente para 20,2 dB, já que os mapas de material passam a ser combinados antes do sombreamento, mas ele se recupera em menos de 2 mil iterações. A partir da inter-reflexão (20 mil), o PSNR sobe para cerca de 28,2 dB em torno de 43 mil iterações; esse ganho de quase 4 dB após a etapa inicial indica que a renderização diferida e o termo indireto respondem por boa parte da qualidade final. Nas últimas milhares de iterações há uma leve queda (de 28,19 para 27,96 dB). Além disso, o script de avaliação mede 27,51 dB sobre as mesmas 200 vistas, 0,45 dB abaixo da medição feita durante o treino; a origem dessa diferença entre os dois scripts ainda será investigada. Esses dois pontos indicam que parte da distância para o artigo pode estar no protocolo de medição e na escolha do checkpoint, e não no método.

Custo computacional. A Tabela III resume o custo por cena. O treino completo da cena *toaster* levou cerca de 1h46min, e a renderização na avaliação atingiu 38,9 FPS. O artigo reporta 0,58 h de treino em média e 122 FPS em uma

NVIDIA A6000. A diferença é coerente com a distância de capacidade entre uma GPU de notebook de 6 GB e uma GPU de estação de trabalho de 48 GB. Ainda assim, mostra que o baseline pode ser treinado integralmente em hardware de consumo, o que viabiliza os experimentos da dissertação.

Análise qualitativa. Na Fig. 2, a renderização reproduz os reflexos especulares do corpo metálico, incluindo os reflexos das fatias de pão sobre a carcaça, uma situação de inter-reflexão. O mapa de normais estimado, por outro lado, é mais suave que o real: os frisos verticais estreitos nas laterais praticamente desaparecem, e surgem artefatos de baixa opacidade no fundo, fora do objeto. Esse descompasso entre aparência correta e geometria aproximada é relevante para as propostas da Seção II. Como a visibilidade do baseline é calculada sobre uma malha derivada dessa geometria, erros finos de normal e de superfície se propagam para o termo de inter-reflexão. Esse efeito motiva tanto a amostragem do lóbulo (P1) quanto a visibilidade calculada diretamente sobre as primitivas (P2).

Próximos passos. (i) Concluir as cinco cenas restantes do Shiny Blender e completar a Tabela II; (ii) estender a replicação ao Glossy Synthetic e ao Shiny Blender Real; (iii) instrumentar o código para medir separadamente o custo da extração de malha e do ray tracing; e (iv) implementar uma primeira versão de P1 com K pequeno e compará-la ao baseline.

V. CONCLUSÃO

Este relatório apresentou o método Ref-Gaussian, adotado como baseline da dissertação, e discutiu duas de suas limitações: a visibilidade testada em uma única direção e a dependência de uma malha extraída periodicamente. Para cada uma foi proposta uma linha de investigação. O baseline foi portado para Windows e executado em uma GPU de notebook, e a replicação na cena *toaster* reproduziu os resultados do artigo com diferença de 0,54 dB em PSNR e métricas perceptuais idênticas.

REFERÊNCIAS

- [1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian splatting for real-time radiance field rendering,” *ACM Trans. Graph.*, vol. 42, no. 4, 2023.
- [2] B. Huang, Z. Yu, A. Chen, A. Geiger, and S. Gao, “2D Gaussian splatting for geometrically accurate radiance fields,” in *Proc. ACM SIGGRAPH*, 2024.
- [3] Y. Yao, Z. Zeng, C. Gu, X. Zhu, and L. Zhang, “Reflective Gaussian splatting,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.
- [4] Y. Jiang, J. Tu, Y. Liu, X. Gao, X. Long, W. Wang, and Y. Ma, “GaussianShader: 3D Gaussian splatting with shading functions for reflective surfaces,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.
- [5] K. Ye, Q. Hou, and K. Zhou, “3D Gaussian splatting with deferred reflection,” in *Proc. ACM SIGGRAPH*, 2024.
- [6] D. Verbin, P. Hedman, B. Mildenhall, T. Zickler, J. T. Barron, and P. P. Srinivasan, “Ref-NeRF: Structured view-dependent appearance for neural radiance fields,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022.
- [7] Y. Liu, P. Wang, C. Lin, X. Long, J. Wang, L. Liu, T. Komura, and W. Wang, “NeRO: Neural geometry and BRDF reconstruction of reflective objects from multiview images,” *ACM Trans. Graph.*, vol. 42, no. 4, 2023.
- [8] B. Karis, “Real shading in Unreal Engine 4,” in *SIGGRAPH Course: Physically Based Shading in Theory and Practice*, 2013.

- [9] N. Moenne-Loccoz, A. Mirzaei, O. Perel, R. de Lutio, J. M. Esturo, G. State, S. Fidler, N. Sharp, and Z. Gojcic, “3D Gaussian ray tracing: Fast tracing of particle scenes,” *ACM Trans. Graph.*, vol. 43, no. 6, 2024.